



LarvaLens

Lap 3

Frontend, Backend and Model Integration

Locked full-stack architecture, typed API contracts, no-mock-data rules, sequential vibe-coding prompts, model checkpoint swaps, integration tests, troubleshooting and deployment guidance.

Prepared for	Mohit S. and the LarvaLens team
Build format	React PWA + FastAPI + Supabase + Leaflet
Version	1.0 - 26 August 2026



Contents

Contents	2
Lap 3 - Product Architecture Before Vibe Coding	3
Lap 3 - Structured Vibe-Coding Prompts	7
Integration Contracts and Acceptance Tests	13
Troubleshooting Guide	17
Source and Dataset References	19

Lap 3 - Product Architecture Before Vibe Coding

5.1 Build one installable PWA, not three client applications

Lap 3 builds one responsive React PWA. It runs in desktop browsers, adapts to phone screens and supports Add to Home Screen. Do not attempt React web + Flutter + Kotlin during the same eight-hour build. Native packaging is a later option after the web product works end to end.

5.2 Locked stack

Layer	Choice	Reason
Frontend	React + Vite + TypeScript	Fast student build, strong mobile responsiveness and PWA support.
Styling	Tailwind CSS + accessible headless components + Lucide icons	Consistent UI without writing a custom design system.
Data fetching	TanStack Query	Explicit loading, retry, invalidation and error states.
Backend	Python 3.11+ + FastAPI + Pydantic	Direct integration with Ultralytics/OpenCV and typed OpenAPI contracts.
Auth/DB/Storage/Realtime	Supabase	Auth, Postgres, private video storage, RLS and job status updates.
Map	Leaflet + OpenStreetMap tiles	Geotagged hotspot visualization without a Google Maps billing dependency.
ML inference	Supplied .pt files loaded by backend lifespan	No external AI and no model creation in the app.
Frontend hosting	Vercel	Static/PWA frontend only.
Inference hosting	Demo laptop or a long-running Python service	Do not depend on a short-lived serverless function for heavy video inference.

5.3 Repository contract

text

```
larvalens/
  apps/
    web/
      src/
        api/
        components/
        features/auth/
        features/scans/
        features/map/
        features/review/
        hooks/
        pages/
        routes/
        types/
      public/
      vite.config.ts
    api/
      app/
        api/v1/
        core/
        db/
        inference/
        schemas/
        services/
        main.py
        models/active/          # gitignored
        tests/
        requirements.txt
      supabase/
        migrations/
        storage_policies.sql
```

```
contracts/
  openapi.json
  model_manifest.schema.json
docs/
  01_PRD.md
  02_TRD.md
  03_APP_FLOW.md
  04_UI_UX.md
  05_BACKEND_SCHEMA.md
  06_IMPLEMENTATION_PLAN.md
.env.example
.gitignore
README.md
```

5.4 Real-time status path

text

```
Web POST /api/v1/scans with JWT + video
-> API validates token, file and model readiness
-> API creates real 'scans' row with status=queued
-> API uploads source video to private storage
-> worker updates status: validating -> detecting -> verifying -> tracking -> completed
-> Supabase Realtime publishes each row update
-> Web progress screen updates from the actual row
-> completed result fetches tracks and signed evidence URL
```

During initial development with no models present, the flow ends at a real error state: `503 MODEL_NOT_READY`. This is deliberate and testable. It prevents the UI from being built around fictional output.

5.5 API contract summary

Method and path	Auth	Behavior
GET /api/v1/health	Public	Liveness only; never exposes secrets or filesystem paths.
GET /api/v1/models/status	Authenticated	Reports required filenames, hashes, ready state and active versions.
POST /api/v1/scans	Authenticated	Multipart video + optional lat/lng; creates a real queued scan; returns 202.
GET /api/v1/scans/{id}	Owner or reviewer	Current real status, progress, result summary and errors.
GET /api/v1/scans	Authenticated	Paginated own history; reviewers may use an explicit queue endpoint.
GET /api/v1/scans/{id}/tracks	Owner or reviewer	Accepted and rejected evidence tracks with reasons.
POST /api/v1/scans/{id}/reviews	Reviewer/admin	Human decision with notes; cannot rewrite raw model evidence.
GET /api/v1/hotspots?bbox=...	Public aggregated or authenticated	Aggregated locations; never exposes a reporter's identity or private video path.
GET /api/v1/review-queue	Reviewer/admin	Scans needing human review.

5.6 Required response behavior

json

```
POST /api/v1/scans -> HTTP 202
{
  "scan_id": "uuid",
  "status": "queued",
  "progress_percent": 0,
  "created_at": "ISO-8601"
}
```

ison

```
GET /api/v1/scans/{id} -> completed
{
  "scan_id": "uuid",
  "status": "completed",
  "progress_percent": 100,
  "probable_larvae_count": 5,
  "rejected_tracks": 7,
  "overall_confidence": 0.86,
  "risk_level": "high",
  "video_quality": "good",
  "model_versions": {
    "detector_sha256": "...",
    "verifier_sha256": "..."
  }
}
```

These are contract examples, not runtime seeds. The application must never insert them.

Lap 3 - Structured Vibe-Coding Prompts

12.1 How to use these prompts

1. Create the six Markdown files exactly as Documents 01-06 above.
2. Start a fresh coding-tool conversation and attach/paste all six documents.
3. Paste Prompt 00 once.
4. Paste Prompts 01-10 one at a time, in order.
5. After every phase, run the requested checks and provide errors back to the same conversation.
6. Do not ask the tool to "build everything" in one prompt; that encourages skipped migrations, fake data and disconnected pages.

12.2 Prompt 00 - permanent source-of-truth guardrail

text

You are implementing LarvaLens. The attached files 01_PRD.md, 02_TRD.md, 03_APP_FLOW.md, 04_UI_UX.md, 05_BACKEND_SCHEMA.md and 06_IMPLEMENTATION_PLAN.md are the source of truth.

Non-negotiable rules:

1. Build one responsive React/Vite/TypeScript PWA and one Python/FastAPI API.
2. Use Supabase Auth, Postgres, private Storage and Realtime; use Leaflet for maps.
3. Do not train, generate, download or substitute any ML model.
4. Do not call Gemini, OpenAI or any external vision API for inference.
5. Lap 1/Lap 2 will supply these required files later:
 - models/active/larva_detector_best.pt
 - models/active/larva_debris_verifier_best.pt
 - models/active/model_manifest.json
 Optional: larva_species_best.pt, disabled by default.
6. Do not add production mock data, seeded scans, random counts, hard-coded predictions, simulated progress or sample hotspots.
7. If model artifacts are absent or their SHA-256 does not match, expose models_ready=false and return HTTP 503 MODEL_NOT_READY from analysis.
8. Frontend must display the real unavailable/error state; it must never replace the error with a demo result.
9. Supabase service-role credentials stay in the API only. RLS is mandatory.
10. Preserve raw model evidence. Human reviews are separate records.

Work phase by phase. Before editing, state which files you will create/change. After editing, provide:

- files changed,
- commands run and exact outcome,
- unresolved errors,
- manual setup still required,
- a phase acceptance checklist.

Do not begin implementation yet. First read all six documents, identify any direct contradiction, and return a concise architecture/folder plan. Do not change the locked stack or invent missing product behavior.

Expected response: architecture review only. Resolve genuine contradictions before Prompt 01.

12.3 Prompt 01 - repository foundation

text

Implement Phase 1 only: repository foundation.

Create the monorepo structure specified in 02_TRD.md:

- apps/web: React + Vite + TypeScript
- apps/api: FastAPI package
- supabase/migrations
- contracts
- docs

Requirements:

- TypeScript strict mode.
- Tailwind and React Router configured.
- TanStack Query and Supabase JS installed.
- API requirements include FastAPI, Uvicorn, Pydantic settings, python-multipart, supabase-py, ultralytics, torch, OpenCV and NumPy.
- Add lint, typecheck and test commands for both apps.
- Add .env.example with variable names from TRD and no values.
- Add .gitignore for secrets, model weights, videos, temporary frames, Python caches, Node outputs and editor files.
- Add a README with exact local start commands.
- Copy the six source documents into docs/ unchanged.

Do not create any business pages, database tables, fake API responses or model stubs in this phase. A health route may return only static service liveness.

Run install/lint/typecheck/test smoke commands that are possible. Stop after the Phase 1 report; do not continue to database/auth.

Acceptance: both processes start; `/api/v1/health` returns liveness; web renders a minimal shell; no secrets/data/model files are tracked.

12.4 Prompt 02 - Supabase schema, RLS and auth

text

Implement Phase 2 only: Supabase database, RLS, private storage definitions and authentication.

Use 05_BACKEND_SCHEMA.md exactly. Create ordered, idempotent migrations for:

- enum types,
- profiles, model_versions, scans, tracks, reviews, scan_events,
- indexes and constraints,
- secure profile trigger,
- has_review_role() helper with fixed search_path,
- RLS policies,
- Realtime publication setup for scans.

Do not let signup metadata assign reviewer/admin. Do not add permissive "authenticated can do everything" policies. Ordinary users must not update raw model result fields, tracks, events or model_versions.

Document private buckets and the server-generated path convention. Do not make videos public and do not put service-role credentials in the web app.

In apps/web implement only real Supabase email/password sign-up, sign-in, sign-out, session hydration, protected-route utility and profile fetch. No mock account and no seeded scan.

Add SQL/manual verification steps for two field workers and one reviewer:

- user A cannot read user B scan,
- field worker cannot insert review,
- reviewer can read review queue evidence,
- signup cannot self-assign admin.

Run migrations if credentials/tooling are available; otherwise provide exact safe commands and do not claim they ran. Stop after the Phase 2 report.

12.5 Prompt 03 - backend contracts and model readiness

text

Implement Phase 3A only: typed backend foundation and model-readiness gate.

Create:

- Pydantic settings with startup validation,
- strict CORS allowlist from APP_ORIGIN,
- request-id middleware,
- structured error response {code,message,retryable,request_id},
- Supabase JWT/current-user dependency,
- repositories for profiles/scans/model_versions,
- model_manifest.schema.json,
- manifest parser and SHA-256 verifier,
- FastAPI lifespan that loads required models once when valid,
- GET /api/v1/models/status.

Required model files are exactly:

larva_detector_best.pt, larva_debris_verifier_best.pt, model_manifest.json.

Optional species model is ignored unless ENABLE_SPECIES_MODEL=true.

If files are missing, malformed or hashes differ:

- API still starts,
- health remains alive,
- models/status returns ready=false with safe reason codes,
- analysis routes will return 503 MODEL_NOT_READY,
- do not expose absolute paths or secret values.

Do not implement fake prediction classes or fallback inference. Tests may create temporary invalid files solely to verify error behavior; they must not generate prediction output. Stop after tests and the Phase 3A report.

12.6 Prompt 04 - real media upload and queued scan

text

Implement Phase 3B only: authenticated real scan creation and private upload.

Add POST /api/v1/scans as multipart/form-data:

- video: required UploadFile,

- latitude, longitude, location_accuracy_m: optional,
- idempotency_key: required UUID/string header or form value.

Before accepting:

1. validate authenticated user,
2. validate models are ready,
3. stream to a bounded temporary file (do not read an unlimited file into RAM),
4. validate allowed MIME/extension, file signature, configured max bytes, OpenCV decodability and configured max duration,
5. create a real scan UUID/path,
6. upload to private scan-videos bucket,
7. create the real scans row,
8. return HTTP 202 with scan_id/status/progress/created_at.

On any failure, clean temporary files and avoid orphan rows/storage objects, or record a compensating cleanup. Use idempotency to avoid duplicate scans on retry. Never generate a count/result in this route.

Also add owner-authorized GET /api/v1/scans/{id} and paginated GET /api/v1/scans. Add tests for missing JWT, wrong owner, model unavailable, oversize, bad MIME, corrupt video, duplicate idempotency key and a valid real test-video path.

Stop after the Phase 3B report.

12.7 Prompt 05 - integrate supplied detector/verifier and tracking

text

Implement Phase 4 only: real inference service using the supplied model files.

Create apps/api/app/inference with clear interfaces:

- ModelRegistry: already-loaded detector/verifier and manifest metadata.
- VideoQualityService: duration, brightness, blur/stability checks.
- InferenceService: frame sampling, detector candidates, 15% padded crops, binary verifier probability, ByteTrack IDs, background global-motion compensation, track persistence, evidence gating and result assembly.

Port the algorithm/thresholds from the approved Lap 2 reference notebook. Read thresholds from model_manifest.json; do not duplicate unexplained constants.

Required output per track:

track_number, detector_confidence, larva_probability, non_larva_probability, motion_score, fused_confidence, persistence_frames, accepted, reject_reason, trajectory and optional evidence frame path.

Required scan output:

probable_larvae_count, rejected_tracks, overall_confidence, risk_level, video_quality, quality_reasons, model_versions.

Rules:

- Species model never decides larva acceptance.
- "No accepted tracks" means count 0, not water-safe.
- Poor quality can yield retake_required instead of an unreliable result.
- Delete temporary frames/crops in finally.
- Do not change model files, train or download anything.

Add unit tests for fusion/gating using explicit numeric track inputs. Those are algorithm tests, not fake production predictions. Add an integration smoke test that is skipped with a clear reason when real model/test-video paths are absent.

Stop after the Phase 4 code/test report; do not build UI pages yet.

12.8 Prompt 06 - job processing, atomic persistence and Realtime

text

Implement Phase 4B only: background processing and real status propagation.

For the hackathon single-process deployment, use a bounded in-process worker queue with one inference worker. Keep the queue behind an interface so a durable worker can replace it later. Document that FastAPI BackgroundTasks alone is not a multi-server durable queue.

Status transitions must be real and monotonic:

queued(0) -> validating(10) -> detecting(30) -> verifying(55) -> tracking(75)
-> completed(100), retake_required(100) or failed(100).

Every transition updates scans and appends scan_events. Persist tracks and final scan result atomically before completed. Store model hashes from the loaded manifest. On exception, write a safe error_code/message and failed status; never expose traceback to the client.

Add owner/reviewer GET /api/v1/scans/{id}/tracks. Add transactional review POST /api/v1/scans/{id}/reviews for reviewer/admin only, preserving model evidence.

Provide the exact Supabase Realtime subscription filter expected by the web app

and a reconnect/refetch rule. Test invalid state transitions, partial write rollback and worker exception handling.

Stop after the Phase 4B report.

12.9 Prompt 07 - frontend shell and real scan journey

text

Implement Phase 5A only: production frontend shell and scan journey.

Follow 03_APP_FLOW.md and 04_UI_UX.md. Build:

- responsive desktop top nav and mobile bottom nav,
- landing, login, forbidden and not-found pages,
- authenticated /scan page,
- VideoCaptureCard with camera/file input and preview,
- client-side size/type/duration checks,
- LocationConsent with Explain, Allow and Skip,
- real multipart API client with JWT and idempotency key,
- model readiness banner,
- explicit 401/413/422/503/network error states.

On 202, navigate only using the returned real scan_id. Do not create a local result object. Preserve the selected file on retry when safe. Never include sample cards, sample history or fake status.

Meet mobile 360px layout, keyboard labels/focus and reduced-motion requirements. Run lint, typecheck and component tests. Stop after Phase 5A report.

12.10 Prompt 08 - processing, result, history and evidence

text

Implement Phase 5B only: actual processing and result pages.

Processing page:

- fetch authorized scan,
- subscribe to Supabase Realtime with filter id=eq.<scanId>,
- show only status/progress received from the row,
- on reconnect, refetch,
- unsubscribe on unmount,
- route completed to result, retain retake/failed with real reasons.

Result page:

- probable count, rejected count, confidence, risk and quality,
- limitation language from PRD,
- evidence funnel using actual counts,
- accepted/rejected TrackEvidenceCards from API,
- reject reason labels,
- review state,
- collapsed model hashes/versions,
- signed evidence media only through an authorized API call.

History:

- paginated real scans for current user,
- loading/empty/error states,
- no seeded rows.

Tests must verify rendering from API fixtures inside the test suite only. Do not ship those fixtures in production code/assets. Stop after Phase 5B report.

12.11 Prompt 09 - map, reviewer and admin status

text

Implement Phase 5C only: privacy-aware map, reviewer flow and model status.

Map:

- Leaflet with OpenStreetMap attribution,
- API endpoint returns aggregated/rounded hotspot cells, never owner ID, raw video path or precise private coordinate,
- filters for time/risk with real empty/error states,
- lazy-load map bundle.

Reviewer:

- role-protected queue and detail routes,
- show raw model evidence read-only,
- confirm/reject/inconclusive + notes,
- write a separate review record,
- do not mutate track probabilities/counts.

Admin model page:

- read-only readiness, filename, SHA-256 prefix, class names and active stage,
- no arbitrary upload/path control in the public UI.

Add authorization tests for field_worker/reviewer/admin. Stop after Phase 5C.

12.12 Prompt 10 - PWA, hardening, deployment and final audit

text

Implement Phase 6/7 only: PWA polish, reliability, deployment configuration and final no-mock audit.

PWA:

- valid manifest and install icons,
- cache application shell/static assets only,
- do not cache authenticated API responses, signed media URLs or raw videos in a shared/public cache,
- provide offline page; optional IndexedDB capture queue only if it is complete and tested, otherwise leave it out.

Hardening:

- loading/empty/error states for every route,
- Realtime reconnect/refetch,
- CORS exact origins,
- private media signed URLs,
- cleanup/temp limits/concurrency guard,
- accessibility pass,
- mobile overflow pass.

Deployment:

- Vercel configuration for apps/web,
- documented persistent/local FastAPI start command,
- production env checklist,
- health/model-status smoke script,
- end-to-end smoke checklist from phone browser.

Audit the entire repository for: mock, demoData, faker, Math.random-based result, hard-coded confidence/count/hotspot, external AI calls, committed secrets, public video buckets and service-role usage in web. Remove production violations; do not remove legitimate deterministic test fixtures.

Run all available lint/typecheck/tests/build. Report exact failures instead of claiming success. Produce a final file list, deployment steps, rollback steps and done-criteria table. Do not add new features.

12.13 Prompt 11 - checkpoint swap without code changes

Use this whenever Lap 1/Lap 2 supplies e100/e200 or rolls back.

text

Do not modify inference logic or thresholds. A new approved active-model folder has been supplied outside Git.

Perform the model-update checklist only:

1. stop the API gracefully,
2. verify model_manifest.json schema,
3. verify required filenames and SHA-256 values,
4. run model load smoke test,
5. run the same positive and negative smoke clips,
6. compare output schema and latency to the previous active version,
7. start the API,
8. verify /health and /models/status,
9. create one real scan and confirm its row stores the new hashes.

If any check fails, restore the previous active folder and report the exact failure. Do not rename a failed checkpoint to best, and do not change app code merely to make an incompatible manifest pass.

Integration Contracts and Acceptance Tests

13.1 Model manifest minimum schema

ison

```
{
  "schema_version": 1,
  "required": {
    "detector": {
      "filename": "larva_detector_best.pt",
      "sha256": "64-lowercase-hex",
      "classes": ["mosquito_larva"],
      "input_size": 640
    },
    "verifier": {
      "filename": "larva_debris_verifier_best.pt",
      "sha256": "64-lowercase-hex",
      "classes": ["larva", "non_larva"],
      "input_size": 224,
      "larva_threshold": 0.70
    }
  },
  "fusion": {
    "detector_threshold": 0.25,
    "min_track_frames": 4,
    "motion_threshold": 0.015,
    "high_morphology_threshold": 0.88
  }
}
```

The backend validates schema and hashes before loading. It does not accept manifest fields from a browser request.

13.2 Risk mapping

Risk is a product-prioritization band, not disease probability.

Condition	Risk label
Count 0	none_observed - only for this clip; not "safe water."
Count 1-2	low
Count 3-5	medium
Count 6+	high

Store the mapping/version in code and document it. A reviewer may change review status but not silently rewrite raw count.

13.3 End-to-end test matrix

ID	Input/condition	Expected system behavior	Owner
E01	Clear video with visible larvae	Real completed scan; accepted tracks and count; model hashes stored.	Lap 3/4
E02	Dirty water with leaves/twigs only	Count 0; candidates may be rejected with non-larva reasons.	Lap 2/4
E03	Reflection/ripples only	No false accepted track or explicit review/retake if uncertain.	Lap 2/4
E04	Camera shake	Compensate global shift or return retake_required; never inflate count silently.	Lap 2/3
E05	Low light/blur	Quality reason and retake_required when threshold fails.	Lap 3/4
E06	Same larva across frames	One unique track/count, not frame-by-frame duplicates.	Lap 2/3
E07	Missing verifier file	503 MODEL_NOT_READY; no upload/result.	Lap 3
E08	SHA mismatch	models/status ready=false; API stays alive.	Lap 3
E09	Oversized/corrupt file	413/422, cleanup done, no fake scan.	Lap 3
E10	User A opens User B scan	Denied without data leakage.	Lap 3/4
E11	Realtime disconnect	Reconnecting indicator then refetch current row.	Lap 3/4
E12	Reviewer decision	Separate review row; raw evidence unchanged.	Lap 3/4
E13	e100 model swap	Manifest/hash change visible; no code change; new scans record new hashes.	Lap 1/2/3
E14	No coordinates	Analysis still completes; map omission explained.	Lap 3/4

13.4 Honest judge-facing metric card

text

Internal prototype evaluation on held-out field clips

Positive clips: __
 Negative clips: __
 Correct positive flags: __ / __
 Correct negative rejections: __ / __
 False alarms: __
 Missed positives: __
 Detector checkpoint SHA: _____
 Verifier checkpoint SHA: _____

Limitations: small prototype benchmark; macro/microscope training data differs from ordinary stagnant-water phone video; further field validation is required.

13.5 Lap 4 ZIP/Git handoff rule

Git stores source, migrations, docs and test code. It should not store large `.pt` files, private videos, `.env` or temporary outputs. Store model versions in the shared model folder with the manifest. If ZIP fallback is needed, use checkpointed names such as `LarvaLens_H3_1530.zip`, never `final_final2.zip`.

Troubleshooting Guide

14.1 Colab runs out of GPU memory

- Keep `batch=-1`; if it fails, set detector batch to 8 or 4.
- Reduce detector `imgsz` from 640 to 512 only after recording the change.
- Close other GPU notebooks; one T4 should not run multiple trainings in parallel.
- Do not enable RAM cache for 8k high-resolution images; `cache="disk"` is safer.

14.2 Classification download count differs

This is expected across dataset versions or because metadata images were excluded. Use the Cell L1-11 accepted, exact-unique count and save its manifest. Check unknown species paths before training.

14.3 YOLO source structure is not discovered

The merger expects image paths containing an `images` directory and corresponding paths under `labels`. If your ZIP is different, inspect one image/label pair and modify only `label_for_image`. Do not rename data blindly until the mapping is confirmed.

14.4 Very high validation score but poor phone video

Likely domain shift or leakage. Check exact/near duplicates, microscope vs field mix, sequential frames across splits, crop scale and lighting. Field-video metrics take priority for the demo claim.

14.5 Verifier labels every crop as larva

Check class balance, negative human review, class-directory names, train/val leakage and threshold. Ensure the loaded file is the binary verifier, not the optional species classifier.

14.6 Count grows every frame

The backend is counting detections rather than unique persistent track IDs. Count only accepted final tracks. Confirm ByteTrack persistence and video frame sampling.

14.7 Realtime UI stays queued

Check backend writes, Realtime publication, RLS visibility, exact scan ID filter and subscription cleanup. Add a refetch fallback; do not simulate progress locally.

14.8 Vercel frontend cannot reach local API

A public HTTPS page cannot reliably call an unreachable laptop URL. For the judging table, use both devices on the same LAN with an appropriate reachable frontend, or expose the persistent API through an approved secure tunnel/host. Keep a local full-stack fallback and a backup processed video.

14.9 Supabase rows exist but user cannot read them

Verify the API created `owner_id` from the authenticated user, JWT is attached, profile exists and RLS policy matches. Do not disable RLS to "fix" the demo.

Source and Dataset References

The following sources informed the implementation choices. Always recheck each dataset's current license/terms before redistribution or commercial use.

1. [Ultralytics - model training configuration, checkpoints, patience and resume concepts](#)
2. [Ultralytics - image classification task and folder-format guidance](#)
3. [Ultralytics - model export formats including ONNX](#)
4. [MosquitoLarvae-7400 Classification dataset](#)
5. [MLMI-2024 dataset record](#)
6. [MLMI-2024 peer-reviewed dataset article, DOI 10.1016/j.dib.2025.111641](#)
7. [IWHR_AI_Lable_Floater_V1 - 3,000-image inland-water floater dataset](#)
8. [IWHR floater dataset benchmark article](#)
9. [AquaTrash dataset](#)
10. [River Floating Debris Dataset](#)
11. [FastAPI - request file uploads](#)
12. [FastAPI - background tasks and the heavy-compute caveat](#)
13. [FastAPI - application lifespan events for model loading](#)
14. [Supabase - Row Level Security](#)
15. [Supabase - Realtime Postgres Changes](#)
16. [Vercel - Python/ASGI runtimes and function limits](#)

15.1 Final one-page execution checklist

Lap 1

- Upload two YOLO ZIPs and run label audit.
- Print actual accepted detection and positive classification counts.
- Train detector e050 -> e100 -> e200, retaining all stages.
- Train optional species model but keep it out of acceptance logic.
- Export/fingerprint and send e050 early.

Lap 2

- Gather 500-1,000 reviewed negatives from public/own sources.
- Keep benchmark clips fully unseen.
- Train balanced binary verifier e050 -> e100 -> e200.
- Run public image and held-out video tests.
- Promote based on false alarms + missed positives, then produce final manifest.

Lap 3

- Paste six documents + Prompt 00.
- Execute Prompts 01-10 sequentially.
- Integrate only supplied manifest-verified files.
- Store real jobs/results and subscribe to real updates.
- Verify RLS, errors, mobile layout and no-mock audit.
- Freeze at Hour 7 and create deployment/backup evidence.

The strongest truthful LarvaLens claim is: "We localize candidates, reject environmental lookalikes, verify persistence across video, count unique tracks and record the exact model versions used - then validate the prototype on held-out positive and negative field clips."